

RulePI: Textual Policy Iteration for Interactive Agents

Anonymous ACL submission

Abstract

An interactive agent can use a failure diagnosis to retry a task, but the lesson is discarded and the extra attempt increases deployment cost. We study a narrow alternative: convert recurring diagnostics into persistent textual rules that a frozen actor can reuse. RulePI performs one policy-iteration step over text: collect training trajectories, diagnose failure modes, propose reusable rules, and optionally retain an update only when held-out return improves. Across ten independently generated TextWorld-24 suites (240 paired test instances), persistent rules improve success from 33.8% to 56.7% (paired 95% CI: +17.1 to +29.2 points). Across ten supported TextArena environments (130 paired test instances), success improves from 19.2% to 61.5% (CI: +18.0 to +72.0 points). Persistent rules attain similar observed success to task-local diagnostic retry while using about half as many test actions. Validation gating does not improve success over ungated persistence. Thus, the evidence supports reusable textual rules for structured actors, not generic language-model self-improvement.

1 Introduction

Interactive agents often receive useful feedback only after they fail. A common response is to append a diagnosis and retry the same task, as in verbal reinforcement and reflection methods (Shinn et al., 2023). This can repair a local failure, but it requires another attempt and does not answer whether the lesson transfers to later tasks. At the other extreme, fine-tuning an actor may be expensive or unavailable.

We examine the middle ground in which the actor is frozen and the policy is editable text. TextGrad showed that natural-language feedback can optimize prompts and compound AI systems (Yuksekgonul et al., 2025). Our question is deliberately smaller and operational: *when the same*

failure mode recurs, can a persistent textual rule replace repeated task-local diagnosis?

RulePI treats a collection of textual rules as a policy. It gathers trajectories, maps failures to candidate rules, and evaluates an updated rule set. The deployed policy is shared across test tasks and receives only one attempt per task. We compare it with (i) a fixed policy, (ii) a strong retry baseline that sees a task-local diagnosis after failure, and (iii) ungated persistence, which stores training-derived rules without validation.

Our contribution is a controlled mechanism study with two findings:

- Persistent rules improve success over a fixed policy on TextWorld and TextArena, with paired intervals above zero.
- Persistence attains similar observed success to diagnostic retry with roughly half the test interaction, but validation gating adds no success benefit.

The second finding is important: the supported claim concerns rule reuse by structured actors, not reinforcement learning of model weights or general LLM self-improvement.

2 Related Work

ReAct interleaves language-model reasoning and actions (Yao et al., 2023), while Reflexion stores verbal feedback to improve subsequent attempts (Shinn et al., 2023). ExpeL extracts natural-language insights from training experiences and recalls them on later tasks (Zhao et al., 2023). ProTeGi uses textual critiques, prompt edits, and search to optimize instructions (Pryzant et al., 2023); TextGrad generalizes this idea to prompts and compound systems (Yuksekgonul et al., 2025). RulePI is closest to ExpeL, but studies a narrower comparison: a reusable text policy versus

a task-local diagnosed retry, with paired evaluation and both test-time and optimization interaction counted.

3 RulePI

Let x be a textual policy and let $\pi(a \mid o, x)$ be a frozen actor that selects action a from observation o . RulePI changes x , not the actor parameters. One iteration has four steps.

1. Collect. Run $\pi(\cdot \mid \cdot, x)$ on training tasks and record trajectories $\tau = (o_t, a_t, r_t)_{t=1}^T$, including success, invalid actions, repetition, and attempt length.

2. Diagnose. Map failed trajectories to a reusable failure mode, such as `graph_exploration` or `objective_sequence`. The current prototype retrieves a human-authored candidate rule for that mode. For example, a repeated room loop yields: “maintain a room graph, prefer unvisited exits, and avoid immediate backtracking.”

3. Persist. Append the candidate rules to form x' . Unlike diagnostic retry, x' is shared by all later tasks rather than discarded after one attempt.

4. Gate (optional). Evaluate old and new policies on disjoint validation instances. RulePI accepts x' when

$$\hat{J}_{\text{val}}(x') \geq \hat{J}_{\text{val}}(x) + \delta, \quad (1)$$

where $\hat{J}$ combines task return, success, validity, and efficiency, and $\delta = 0.001$. TextArena additionally requires the paired-bootstrap lower bound on improvement to be nonnegative. The ungated ablation always keeps the training-derived update.

What is being optimized, and where is the RL?

The positive experiments use prompt-aware structured actors. They recognize rule content and activate existing routines for objective sequencing, graph exploration, recipe execution, or game strategy. This makes the experiment easy to inspect: the outer loop selects and reuses a capability; it does not invent that capability or train neural weights. The environment supplies trajectories and scalar return, and the validation comparison is a discrete policy-improvement step. RulePI is therefore textual policy selection under reward, not a policy-gradient or PPO algorithm.

4 Experimental Setup

TextWorld-24. TextWorld generates controlled text environments with varied maps and quests (Côté et al., 2018). Each repetition contains six instances from each of four families: simple objectives, coin collection, treasure hunting, and cooking. For every repetition, 8 training instances, 8 validation instances, and 24 test instances use disjoint seeds. We run 10 independent repetitions, yielding 240 paired test instances per method, with at most 80 actions per attempt.

TextArena-10. We use ten environments for which the actor implements the corresponding rule interface: Nim, Connect Four, Reverse Tic-Tac-Toe, Guess the Number, Frozen Lake, Tower of Hanoi, Lights Out, Mastermind, Blackjack, and Bandit (Guertler et al., 2025). Three seeds per environment are used for training and validation; ten fresh test seeds and both player sides where applicable yield 130 paired test instances per method.

Baselines. FIXED uses the initial policy once. RETRY+DIAG first uses the fixed policy; after failure it receives the task’s diagnosed rule and retries once, so its maximum test budget is 160 actions in TextWorld. UNGATED learns persistent rules from training trajectories and retains them without validation. RULEPI applies the validation gate in Section 3 and then freezes the policy for test.

Statistics. Methods are paired on identical test instances, including seeds and target sides. We report hierarchical paired-bootstrap 95% intervals from 10,000 resamples, clustering by TextWorld generation or TextArena environment and resampling test instances within clusters. Optimization actions include environment steps used to produce and evaluate the policy; test actions include both attempts for RETRY+DIAG.

5 Results

5.1 Persistent Rules Improve Success

Table 1 supports the first claim. On TextWorld, RulePI improves success by 22.9 percentage points over the fixed policy; the paired interval excludes zero. On TextArena, the gain is 42.3 points and again excludes zero. Diagnostic retry and persistent rules are close in success: RulePI is 1.25 points higher on TextWorld (CI: 0.0 to 3.3) and tied on TextArena.

Benchmark	Fixed	Retry+Diag	Ungated	RulePI	RulePI – Fixed (95% CI)
TextWorld-24 (240 instances)	33.8	55.4	56.7	56.7	+22.9 [17.1, 29.2]
TextArena-10 (130 instances)	19.2	61.5	61.5	61.5	+42.3 [18.0, 72.0]

Table 1: Success rates (%) for matched test instances. Intervals are hierarchical paired-bootstrap intervals for RulePI minus the fixed policy.

Family (60 instances each)	Fixed	Retry	RulePI
Coin collection	100.0	100.0	100.0
Cooking	0.0	16.7	16.7
Simple objective	0.0	11.7	11.7
Treasure hunting	35.0	93.3	98.3

Table 2: TextWorld success (%) by task family.

Benchmark	Test actions			N^*
	Retry	RulePI	Opt. actions	
TextWorld	81.78	42.25	7,965	202
TextArena	20.68	10.19	7,921	756

Table 3: Mean actions per deployed task, one-time RulePI optimization actions, and interaction break-even count N^* .

Method	Attempts	Actions	Outcome
Fixed	1	80	loop; failure
Retry+Diag	2	86	success
RulePI	1	6	success

Table 4: Matched test example. Retry+Diag uses 80 failed actions followed by the same six-action route as RulePI.

Where do the TextWorld gains occur? Table 2 shows that RulePI improves three families and leaves the already-solved coin family unchanged. The largest gain is on treasure hunting, where the exploration rule directly addresses repeated room loops. Cooking and simple objectives remain difficult, which prevents the overall result from being read as universal task solving.

5.2 Persistence Reduces Deployment Interaction

Table 3 supports the second claim. RulePI uses one test attempt and about half as many test actions as diagnostic retry: 42.25 versus 81.78 on TextWorld and 10.19 versus 20.68 on TextArena. The paired reductions are 39.53 actions (CI: 34.17 to 44.95) and 10.48 actions (CI: 7.35 to 14.19), respectively.

This is an amortization claim, not a free-compute claim. If C_{opt} is optimization interaction and $A_r - A_p$ is the per-task saving over retry, the break-even deployment count is $N^* = C_{\text{opt}} / (A_r - A_p)$. RulePI crosses this point within the 240-instance TextWorld study, but not within the 130-instance TextArena study.

5.3 A Matched Example

Table 4 traces one logged test instance, `treasure_level_15`. The fixed policy

repeats the same four-direction loop until the 80-action horizon. Both improved methods then execute the same six-action route, but Retry+Diag first pays for the failed attempt. RulePI learned the reusable exploration rule only from disjoint training instances and applies it on its first test attempt.

5.4 What the Evidence Does Not Show

The gate does not improve success. Gated and ungated persistence are identical on TextWorld success, reward, and test actions. On TextArena they tie in success and test actions; gating retains 6 rather than 10 rules and changes mean reward only slightly (0.573 versus 0.564). Thus, persistence explains the observed success gain; validation is only a conservative diagnostic in these runs.

6 Practical Guidance

RulePI is useful when (i) many future tasks share failure modes, (ii) the actor reliably executes written rules, and (iii) the policy will be reused enough to amortize evaluation. For a small number of tasks, diagnostic retry is simpler and can cost less overall. A practical deployment should first measure the action saving $A_r - A_p$, estimate N^* , and retain a gate only if rejected updates or validation regressions occur in practice.

7 Conclusion

Persistent textual rules improve two controlled interactive-agent suites and attain comparable observed success to task-local diagnostic retry with substantially less deployment interaction. The clean result is rule reuse, not a benefit from val-

idation gating and not LLM self-improvement. This narrower conclusion makes RulePI an interpretable testbed for studying when trajectory lessons should become shared policies.

Limitations

The positive experiments use structured actors that detect human-authored rule content and dispatch pre-existing routines. Candidate rules are selected from a predefined library keyed by benchmark failure modes; they are not freely generated from raw trajectories. Consequently, the study measures rule selection and reuse, not the acquisition of new skills or general natural-language instruction following. The task-local retry baseline receives the same diagnosed rule after its first failed test attempt, while persistent methods learn only from training data; this is intentionally a strong deployment-time baseline but uses a larger test budget. We do not provide an adequately powered evaluation with a generative language-model actor, so the structured-actor results should not be interpreted as general LLM prompt optimization. TextWorld rejected some requested generation seeds; the runner deterministically advanced to the next valid seed and logged requested and actual seeds before evaluation. Finally, interaction counts measure environment actions, not wall-clock time, energy, or textual-optimizer inference cost.

Ethical Considerations

The experiments use synthetic environments and no personal or sensitive data. Persistent agent rules can nevertheless amplify misspecified rewards or unsafe behavior; deployments should log rule provenance, validation decisions, and regressions, and should restrict which rules can be activated in consequential environments.

References

- Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Ruoyu Tao, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. 2018. [TextWorld: A learning environment for text-based games](#). *Computing Research Repository*, arXiv:1806.11532.
- Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. 2025. [TextArena](#). *Computing Research Repository*, arXiv:2504.11442.

- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. [Automatic prompt optimization with “gradient descent” and beam search](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968. Association for Computational Linguistics.

- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652.

- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language models](#). In *International Conference on Learning Representations*.

- Mert Yuksekogonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. [Optimizing generative AI by backpropagating language model feedback](#). *Nature*, 639:609–616.

- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2023. [ExpeL: LLM agents are experiential learners](#). *Computing Research Repository*, arXiv:2308.10144.

A Reproducibility Details

Terminology. **Game/environment** is the underlying task definition, such as a generated TextWorld world or TextArena’s Nim environment. **Test instance** is a specific task determined by seed and, for two-player TextArena environments, target side. **Attempt/rollout** is one interaction from reset until success, failure, or the action limit. **Action/step** is one move within an attempt. For example, TextWorld `treasure_level_15` at one generation seed is one test instance; retrying it creates a second attempt, not a second instance. For TextArena, `Nim-v0` at one seed with target side 0 and target side 1 yields two test instances. We evaluate 240 matched TextWorld test instances and 130 matched TextArena test instances. An attempt is one rollout on an instance; retry methods may use two attempts.

All four methods are evaluated on identical test instances. For TextWorld, requested outer seeds are spaced by 100,000; generator-invalid requests are advanced by one and recorded in a manifest. Within each repetition, training and validation seeds are offset by 10,000 and 20,000 from the test seed. The textual optimizer adds at most four rules in TextWorld and evaluates candidate policies under the same 80-action horizon used at

test. The three reusable TextWorld rules cover ordered objective execution, graph exploration, and cookbook-based recipe completion.

B Diagnosis and Rule Libraries

Automated diagnosis. No LLM or human annotator labels trajectories during either experiment. TextWorld applies three deterministic predicates: a failed or repeated simple, coin, or treasure trajectory proposes `objective_sequence`; a failed cooking trajectory proposes `cooking_recipe`; and any repeated trajectory or attempt longer than 20 actions proposes `graph_exploration`. TextArena groups training trajectories by environment and creates an assignment when reward is below 1, an action is invalid, or an action has negative estimated advantage. It labels the cause `invalid_action`, `long_horizon_credit`, `repeated_action`, or `environment_strategy` using those logged fields, then retrieves the rule keyed by environment ID. These criteria are implemented without free-form text classification.

Human involvement. The human-authored assets are 3 TextWorld rule sentences (78 words, 581 characters) and 12 TextArena candidate sentences (175 words, 1,111 characters): 10 environment rules and 2 generic rules. The structured actors also contain three TextWorld rule-triggered routines and one strategy routine for each of the 10 TextArena environments. There is no per-trajectory annotation and no post-hoc manual selection. We did not record authoring time, so we cannot provide a defensible person-hour estimate; the reproducible unit of engineering effort is one rule sentence and one pre-existing strategy interface per supported capability. The two generic TextArena candidates do not activate an additional structured-actor branch.

Complete library. The exact TextWorld rules are:

Rule ID	Full rule text
<code>objective_sequence</code>	For TextWorld objective games, convert the visible objective into ordered subgoals: movement directions, open/unlock commands, take target objects, and put/place target objects on or in the requested destination.
<code>graph_exploration</code>	For TextWorld exploration, maintain a room graph, prefer unopened containers and unvisited exits, and avoid immediate backtracking unless no other admissible progress action exists.
<code>cooking_recipe</code>	For TextWorld cooking, search for the kitchen/cookbook, take and read the cookbook, collect listed ingredients, follow recipe directions for cutting/cooking, then prepare and eat the meal.

The exact TextArena rules are:

Rule ID	Full rule text
Nim	For Nim, use the xor strategy: move to a zero nim-sum when possible.
Connect Four	For Connect Four, take an immediate winning column and otherwise block an opponent immediate connect-four.
Reverse Tic-Tac-Toe	For Reverse Tic Tac Toe, avoid moves that immediately complete your own three-in-a-row.
Guess the Number	For Guess The Number, maintain lower and upper bounds from higher/lower hints and guess the midpoint.
Frozen Lake	For Frozen Lake, plan a shortest safe path to the goal while avoiding holes.
Tower of Hanoi	For Tower of Hanoi, follow the recursive optimal disk-moving plan from A to C.
Lights Out	For Lights Out, solve the binary toggle system and press cells from a valid solution.
Mastermind	For Mastermind, maintain all candidate codes and eliminate candidates inconsistent with black/white feedback.
Blackjack	For Blackjack, use the basic threshold policy: hit below 17 and stand at 17 or higher.
Bandit	For Bandit, explore every button once, track empirical reward, then exploit the best observed button.
Generic validity	Use only legal bracketed actions from the current observation, and avoid actions that just produced invalid-move feedback.
Generic state	Track the latest visible game state after every observation before choosing the next action.

C Additional Controls

We evaluate controls on the same matched test instances as the main paper. ALWAYS-ON enables the complete library before any trajectory is observed. RANDOM-6 averages 30 independently sampled sets of six TextArena environment rules; it matches RulePI’s six active environment rules but ignores diagnostics. RETRY+PERSIST processes tasks in the benchmark’s fixed order, retries a failed task once with its diagnosed rule, and retains that rule for all later test tasks.

TextWorld-24	Success	Attempts	Actions
RulePI	56.7	1.00	42.25
Always-on (3)	56.7	1.00	42.25
Retry+Persist	56.7	1.48	78.94

TextArena-10	Success	Attempts	Actions
RulePI (6)	61.5	1.00	10.19
Always-on (12)	61.5	1.00	10.78
Random-6	42.8	1.00	11.19
Retry+Persist	61.5	1.43	17.28

Control results; success is in percent.

TextWorld’s always-on result is identical to RulePI because every repetition diagnoses all three available rules; TextWorld therefore isolates the value of the rule-enabled controller, not trajectory-driven selection among rules. On TextArena, random selection is substantially worse than trajectory-selected RulePI, while always-on ties success but uses 0.59 more actions per task. Thus, TextArena supports a selection benefit; TextWorld does not. Retry+Persist reaches the same observed success but remains more expensive at deployment than a policy learned on disjoint training tasks. Random-6 ranges from 30.8% to 53.8% across 30 policies (SD 6.7 points); each policy reuses the same 130 test instances, so the resulting 3,900 one-attempt evaluations are not independent test instances.

D Gate, LLM, and Iteration Diagnostics

Gate sensitivity. We recompute TextWorld acceptance post hoc from the saved eight-instance validation trajectories. The default score is reward +0.5 success −0.4 invalid −0.1 repeated −0.004 actions. The deployed test result selects the saved candidate or fixed-policy trajectory according to each counterfactual gate.

Gate	Accepted	Success	Actions
Default, $\delta = 0.001$	10/10	56.7	42.25
Default, $\delta = 0.05$	9/10	54.2	42.39
Default, $\delta = 0.20$	6/10	46.2	43.35
Success-only, $\delta = 0.001$	10/10	56.7	42.25
Reward-only, $\delta = 0.001$	9/10	54.2	42.39
Family-protected	7/10	49.2	42.80

The original gate prevents no update because all 10 candidates pass. A protected-regression variant requiring nonnegative validation change in every family rejects three updates, but whole-policy rejection also discards gains in other families and reduces aggregate test success to 49.2%. We did not run larger validation sets, and these post-hoc comparisons are not fresh confirmatory experiments. They show that stricter gates change acceptance, not that gating improves held-out success.

Further iterations and conflicts. The experiment performs one iteration. With the fixed three-rule TextWorld library, a deterministic second pass proposes only normalized duplicates: the policy remains at three rules and version 1, so no behavioral improvement is possible without new candidates. Larger libraries and freely generated rules were not tested. Exact duplicates are suppressed; TextArena environment rules occupy separate variables; and validation chooses among candidate edits. There is no semantic conflict resolver beyond validation and prompt-length limits, so overfitting and contradictory-rule behavior remain open questions.